AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY

Amrita School of AI
COIMBATORE CAMPUS

EXERCISE 11 · UNIT 2

Domain decomposition with MPI halo exchange

Split a domain across ranks, exchange halos without deadlocking, and get the same answer on any number of processes including ones that do not divide the problem evenly.

Prepared by **Dr. Abhijith Anandakrishnan**
Assistant Professor, Amrita School of AI

EXERCISE	UNIT
ex11-mpi-halo-stencil	2 — OpenMP and MPI
LANGUAGE	MARKS
C, MPI	10
SUBMIT AS	OFFERING
<ROLLNO>.c	Odd Semester 2026-27

The problem

A metal bar is held at 100 degrees at one end and 0 at the other. Heat diffuses along it. Discretise the bar into n cells and repeatedly replace each cell with the average of its two neighbours:

$$u_{\text{new}}[i] = \tfrac{1}{2} \left(u[i-1] + u[i+1] \right)$$

This is **Jacobi iteration** on a one-dimensional stencil, and it is the smallest honest example of the pattern behind most distributed scientific computing.

Your job is to run it across P MPI ranks and get the right answer for **any** P.

What you must implement

The starter file contains the harness: argument parsing, a serial `reference_sum()` you can check yourself against, and the verification block that prints the result. Four things are missing, marked `TODO`.

1. Decomposition. Split *n* interior cells across *size* ranks. `n` is not always divisible by `size`.

> **PITFALL · THE UNEVEN CASE IS WHERE MOST SUBMISSIONS FAIL**
>
> `local_n = n / size` silently loses `n % size` cells, and the answer comes out slightly wrong on exactly the rank counts nobody tests with. Give the first `n % size` ranks one extra cell each. Then every rank's share differs by at most one, and the shares add up to exactly *n*. The grader tests with 7 ranks for this reason.

2. Halo cells. Allocate `local_n + 2` doubles so that interior indices are `1 .. local_n` and indices `0` and `local_n+1` are the **halo**: copies of the neighbouring rank's edge value.

The physical boundaries belong in the halo slots of the end ranks: rank 0 holds `LEFT_BC` at index 0, and the last rank holds `RIGHT_BC` at index `local_n+1`.

3. The time loop with non-blocking exchange. Each step:

```
a. post MPI_Irecv for both halos
b. post MPI_Isend of both edge values
c. update the interior cells that do not need a halo   <- real work, overlapped
d. MPI_Waitall
e. update the two edge cells
f. swap u and v
```

4. Global sum. Sum your rank's interior cells and `MPI_Reduce` them to rank 0.

Why non-blocking, and why the grader insists

The obvious exchange deadlocks:

```
MPI_Send(&u[1], 1, MPI_DOUBLE, left,  0, comm);   /* every rank sends first */
MPI_Recv(&u[0], 1, MPI_DOUBLE, left,  0, comm, &st);
```

`MPI_Send` is permitted to block until a matching receive is posted. Every rank is waiting for a receive that nobody has reached.

COMMON MISCONCEPTION · IT WORKED, SO IT MUST BE FINE

With one double per message it will almost certainly appear to work, because small messages are copied into an internal buffer and `MPI_Send` returns immediately. Widen the halo to a 2-D face of a few hundred kilobytes and the implementation switches to a rendezvous protocol that genuinely blocks. The same code then deadlocks on the day you scale it up.

The grader rejects `MPI_Send` in this exercise so that you learn the habit now rather than in a production run.

Non-blocking also buys the thing that actually matters at scale: while the messages are in flight, the interior cells (which need no halo) can be updated. The network cost disappears behind real work.

DEFINITION · MPI_PROC_NULL

A rank that does not exist. Sends to it and receives from it succeed immediately and do nothing.

`c int left = (rank == 0) ? MPI_PROC_NULL : rank - 1; int right = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;`

Every boundary special case disappears, and the time loop has no `if` in it.

Tag discipline

A receive matches a send only when source, tag and communicator all agree. In a symmetric exchange you have two messages travelling in opposite directions between the same pair of ranks, so they need different tags:

```
MPI_Irecv(&u[0],           1, MPI_DOUBLE, left,  0, comm, &req[0]);
MPI_Irecv(&u[local_n + 1], 1, MPI_DOUBLE, right, 1, comm, &req[1]);
MPI_Isend(&u[1],           1, MPI_DOUBLE, left,  1, comm, &req[2]);
MPI_Isend(&u[local_n],     1, MPI_DOUBLE, right, 0, comm, &req[3]);
```

Note the pairing: what you send left with tag 1 is what your left neighbour receives from its right with tag 1.

PITFALL · THE BUFFER IS NOT YOURS UNTIL WAITALL RETURNS

After `MPI_Isend` the message has not been sent. Overwriting `u[1]` before `MPI_Waitall` completes corrupts the message. This is why the edge cells are updated **after** the wait, and the interior cells before it.

Building and running

```
mpicc -O2 -std=c11 -Wall -Wextra -Werror -Iinclude stencil.c -o stencil -lm

mpirun -np 1 ./stencil 240 500
mpirun -np 4 ./stencil 240 500
mpirun --oversubscribe -np 7 ./stencil 240 500     # more ranks than cores
```

Every run must print `RESULT OK`. The program checks itself against the serial reference, so you do not need to compare anything by hand.

ON ASAICOMPUTE · RUNNING IT PROPERLY ON ASAICOMPUTE

`mpirun` on a laptop puts every rank on one machine, which tests correctness but not communication. On the cluster:

```bash
#!/bin/bash
```

SBATCH --job-name=ex11

SBATCH --nodes=2

SBATCH --ntasks-per-node=8

SBATCH --time=00:05:00

srun ./stencil 4000000 2000 ```

Run it at 1, 2, 4, 8, 16 and 32 ranks and plot the speedup. You will see it flatten, then fall. The communication per rank stays constant while the computation per rank shrinks, so the ratio worsens with every rank you add. That is Amdahl’s law appearing in your own measurement, and it is a good figure for your report.

How this is marked

CHECK	MARKS	WHAT IT MEANS
Compiles cleanly	1	Builds with `-Wall -Wextra -Werror`
Uses non-blocking communication	2	`Irecv`, `Isend` and `Waitall` present, `MPI_Send` absent
Correct on 1 rank	2	Degenerate case: no neighbours at all
Correct on 4 ranks	3	Even decomposition
Correct on 7 ranks	2	Uneven decomposition, 240 = 7×34 + 2

Submitting

Rename your finished file to your roll number and submit that one file:

```
AIE23001.c
```

NOTE · BEFORE YOU SUBMIT

Run it at 1, 4 and 7 ranks. If any of the three does not print `RESULT OK`, the grader will find the same thing. Seven is the one people forget.